

Part 2

Architecture and Pipeline

How LoopHole is put together - system context, the four-stage pipeline, the complete data model, a fully worked matmul example with real intermediate values, the lifter's decision logic, configuration, and the verification, emission and toolchain architectures with their structural weaknesses.

AUDIENCE

Anyone who will change code or needs to reason about what a result means.

COVERS

system context, pipeline stages and contracts, data classes and enums, matmul walkthrough, result states and exit codes, policy profiles and environment variables, Z3 encoding strategies, emitter design, frontend and validation paths, module dependency graph, architectural drawbacks

REPOSITORY STATE

branch main lineage, commit 0390e08

GENERATED

2026-09-14

SOURCE

docs/handbook/source/02-architecture.md

Contents

System Context	4
What the tool is not	5
The Four-Stage Pipeline	6
Data Model	8
Parser output (affine_extractor.py)	8
Sketch library (sketch_library.py)	9
Tracer output (sympy_tracer.py)	9
Verifier output (z3_checker.py)	9
Lifter output (lifter.py)	10
Frontend, validation, replay and corpus types	10
Worked Example: Matrix Multiply End to End	11
Input	11
Stage 1: extraction	11
Stage 2: candidate matching	11
Stage 3: verification	12
Stage 4: emission	12
The same kernel from real C code	13
Lifter Decision Flow and Result States	14
Candidate verification order	14
Acceptance and exit codes	15
Configuration: Profiles, Flags and Environment	16
Policy profiles	16
Environment variables	16
Verification Architecture	17
Overall encoding	17
How the right-hand side is built	18
Reduction strategies	18
Dimension alignment	18
Parametric proof pilot (A-15)	19
Numeric semantics	19

Emission Architecture	20
Handler tables	. 20
Types and semantics	. 20
Validity	. 20
The missing link between sketch and emitter	. 20
Frontend and Validation Architecture	21
Two separate paths into Polygeist	. 21
Verifier discovery	. 21
Module Dependency Graph	22
Architectural Drawbacks Summary	23

System Context

LoopHole is a single Python package (`packages/loophole`) with a command-line interface. It relies on three kinds of external component:

1. **Python libraries** installed on the host: `z3-solver` (SMT solver), `sympy` (algebra), `click` (CLI framework) and `rich` (terminal tables and colours). They are declared in `pyproject.toml`.
2. **A Docker image** (`loophole-polygeist:llvm17`, also tagged `ghcr.io/schizoid-man/loophole-polygeist:llvm17`) containing LLVM/Clang/MLIR 17 from Ubuntu packages and a `cgeist` binary built from Polygeist. The host never needs LLVM installed; the CLI shells out to `docker run` whenever it needs `cgeist` or `mlir-opt`.
3. **GitHub infrastructure**: Actions for tests and image publishing, and the GitHub Container Registry (private) for the prebuilt image.

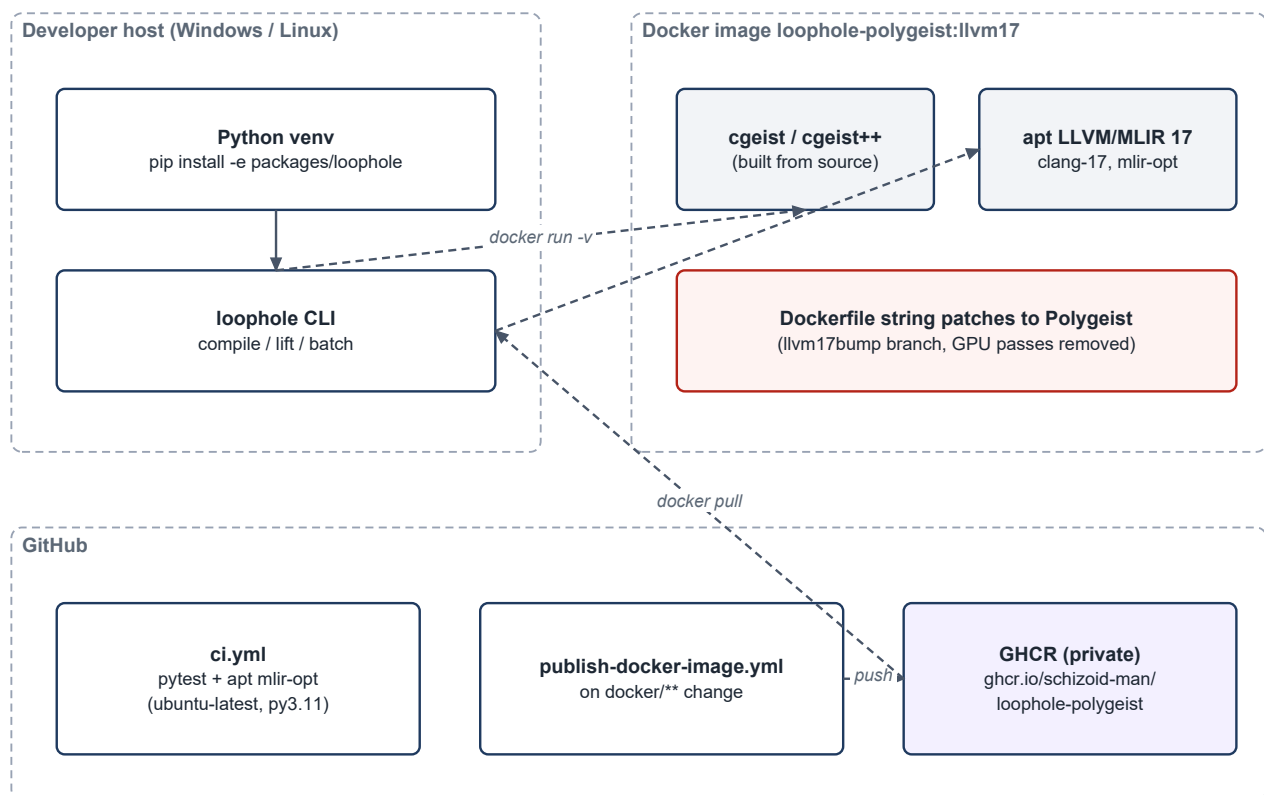


Figure: where each piece runs. The host only needs Python; LLVM 17, `cgeist` and `mlir-opt` live in the Docker image, which CI publishes to GHCR.

The inputs to the system are C/C++ source files or MLIR text files. The outputs are MLIR text files in the Linalg or StableHLO dialect, terminal reports, and JSON/Markdown reports. There is no server, database or persistent state: every command is a stateless batch process, which keeps the operational picture simple.

What the tool is not

- It is not an MLIR pass or plugin; it never links against MLIR and never builds MLIR objects in memory. Everything is text in, text out.
- It is not a general synthesiser: it can only produce the 35 operations in its sketch library, and only for loop nests whose shape matches one of them exactly (same number of loops, same parallel/reduction split).
- It does not run the lifted code or compare numbers. Downstream compilation (IREE, XLA) was exercised once by hand (see the lane A full-workflow log) but is not part of the tool or CI.

The Four-Stage Pipeline

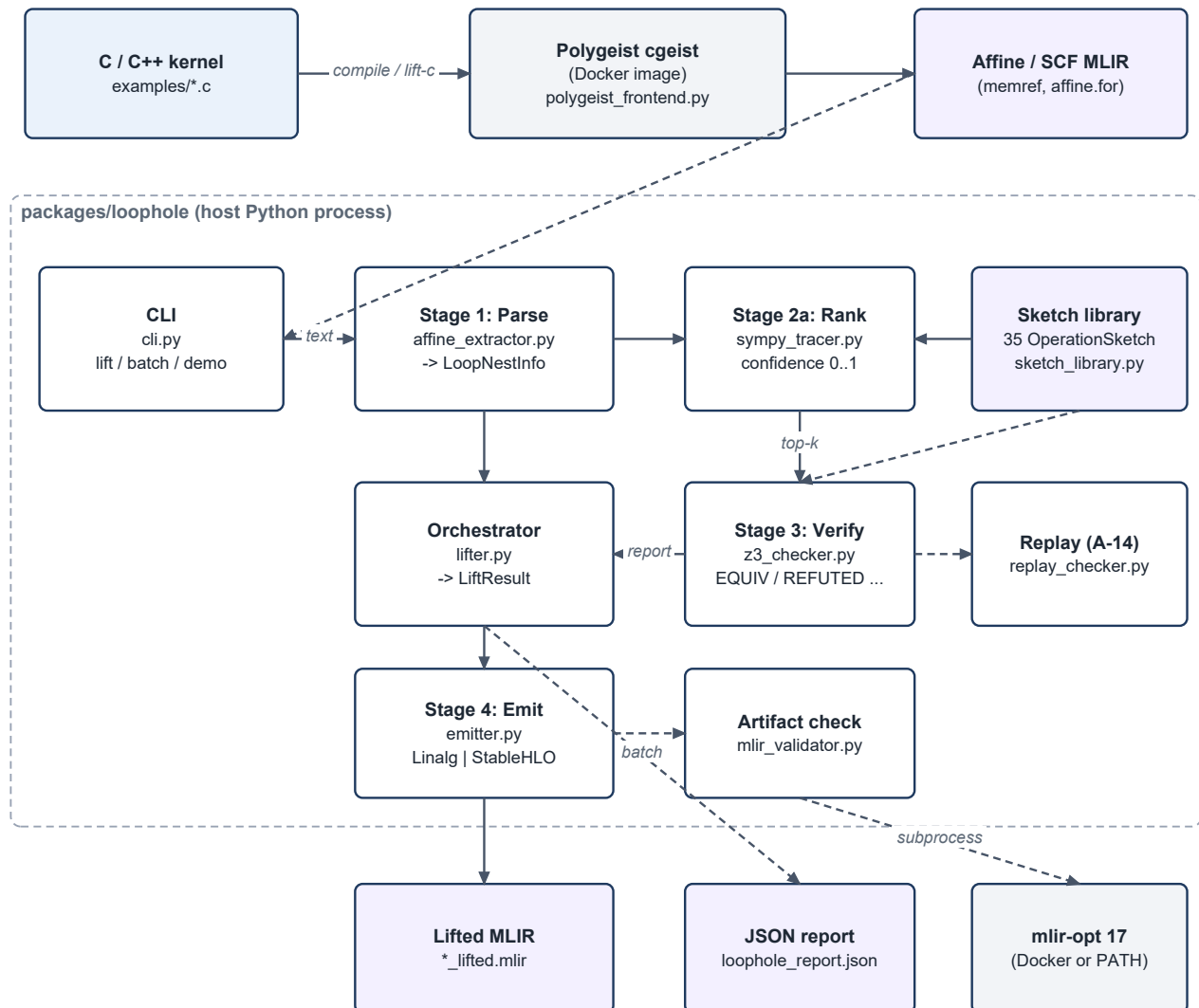


Figure: end-to-end data flow. Solid arrows are the main path; dashed arrows are optional or out-of-process steps.

All lifting goes through `Lifter.lift(mlir_text)` in `lifter.py`. The stages, with their contracts:

Stage	Module and entry point	Input	Output	Failure behaviour
0 (optional) Frontend	<code>polygeist_frontend.PolygeistFrontend.generate_mlir</code> , or <code>cli.compile_cmd</code>	C/C++ files	MLIR text	Raises <code>PolygeistFrontendError</code> (CLI prints a panel and exits 2)
1 Parse	<code>affine_extractor.AffineExtractor.extract</code>	MLIR text (one function expected)	<code>LoopNestInfo</code>	Almost never raises; records <code>ParsingWarning</code> diagnostics and returns partial data (for example <code>func_name="unknown"</code>)
2 Match	<code>Lifter._match_candidates</code> using <code>structural_match</code> and <code>SympyTracer</code>	<code>LoopNestInfo</code>	up to <code>2*top_k</code> (sketch, confidence) pairs	Empty list leads to a result with <code>verification=None</code>
3 Verify	<code>Lifter._verify_candidates</code> using <code>Z3EquivalenceChecker.check</code>	<code>LoopNestInfo</code> , candidates	best (sketch, <code>VerificationReport</code> , confidence) or <code>None</code>	<code>ENCODE_ERROR</code> and <code>STRUCTURAL_MISMATCH</code> reports are dropped

Stage	Module and entry point	Input	Output	Failure behaviour
4 Emit	<code>LinalgEmitter.emit</code> or <code>StableHLOEmitter.emit</code> via <code>Lifter._emit</code>	sketch, <code>LoopNestInfo</code>	MLIR text	<code>EmissionError</code> becomes <code>LiftResult.error</code>
5 (optional) Validate	<code>mlir_validator.validate_mlir_artifact</code> (only called by <code>batch</code>)	MLIR text	<code>MlirValidationResult</code>	Non-zero exit of the verifier marks failure; batch exits 1 at the end

A crucial property of this design is that **stages communicate only through `LoopNestInfo` and `OperationSketch`**. Whatever the parser does not put into `LoopNestInfo` (loop steps, operand order, which value flows where, which function a loop belongs to) is invisible to every later stage. Most of the soundness problems in Part 5 are consequences of information that never makes it into this structure, or that is present but ignored by the verifier.

Data Model

All data classes are plain `@dataclass` objects with no methods beyond small derived properties. They are defined in the module of the stage that produces them.

Parser output (`affine_extractor.py`)

`AccessPattern`: one `affine.load/memref.load` (read) or `affine.store/memref.store` (write).

Field	Type	Meaning
<code>tensor_name</code>	str	SSA name of the array, with %, for example %A
<code>index_exprs</code>	List[str]	Normalised index expressions using short IV names, for example ["i", "k"] or ["2*oh + 3*kh", "ow + 2*kw"]
<code>is_affine</code>	bool	True for <code>affine.*</code> , False for <code>memref.*</code>
<code>is_read</code>	bool	True for loads
<code>shape</code>	List[int]	Shape parsed from the type annotation on this access; -1 for dynamic
<code>element_type</code>	str	For example <code>f32</code>
<code>ssa_name</code>	str	For loads the result (<code>%a</code>); for stores the <i>stored value</i> (<code>%add</code>)

`ComputeOp`: one `arith.*` operation.

Field	Type	Meaning
<code>op_type</code>	str	Suffix after <code>arith.</code> , for example <code>mulf</code> , <code>addf</code> , <code>constant</code>
<code>operands</code>	List[str]	Operand SSA names, truncated to two
<code>result</code>	str	Result SSA name
<code>value</code>	Optional[float]	For constants; 0.0 when unparseable

`ParsingWarning`: diagnostic with `level` (`debug/warning/error`), `location` (parser method name), `reason`, `input_snippet` (up to 80 characters) and optional `guidance`.

`LoopNestInfo`: the single contract between the parser and everything else.

Field	Type	Meaning
<code>func_name</code>	str	From the first <code>func.func @name(</code> in the text
<code>induction_vars</code>	List[str]	Short IV names in source order
<code>bounds</code>	Dict[str, (lo, hi)]	Integers when resolvable, else strings such as <code>N</code> or unresolved text
<code>loop_order</code>	List[str]	Same as <code>induction_vars</code> today
<code>reads, writes</code>	List[AccessPattern]	In textual order across the whole input
<code>compute_ops</code>	List[ComputeOp]	In textual order
<code>tensor_shapes</code>	Dict[str, List[int]]	Reconciled shape per tensor (majority rank, conflicting static dims made dynamic)
<code>tensor_types</code>	Dict[str, str]	Dominant element type per tensor
<code>element_type</code>	str	Most common element type overall, default <code>f32</code>
<code>reduction_vars, parallel_vars</code>	List[str]	Classified by whether the IV name occurs in any write index
<code>func_args</code>	Dict[str, str]	Argument name to type string
<code>has_accumulation, accumulation_op</code>	bool, str	Output tensor both read and written, with an add

Field	Type	Meaning
<code>diagnostics</code>	<code>List[ParsingWarning]</code>	Collected during extraction
<code>output_tensor</code> (property)	<code>Optional[str]</code>	Tensor of the first write
<code>input_tensors</code> (property)	<code>List[str]</code>	Read tensors other than the output, with duplicates

Notably absent: loop steps, per-loop parent/child structure, which operations are inside which loop, `iter_args` and `affine.yield` links, conditionals, and any notion of multiple functions.

Sketch library (`sketch_library.py`)

`IteratorType` (str enum): `PARALLEL`, `REDUCTION`, `WINDOW` (unused).

`ComputePayloadType` (str enum): `MULTIPLY_ACCUMULATE` (`C += A*B`), `COPY`, `ADD`, `SUBTRACT`, `MULTIPLY`, `MAX`, `MIN`, `RELU`, `SCALE`, `NEGATE`, `ACCUMULATE_ADD` (`C += A`), `ACCUMULATE_MAX` (`C = max(C, A)`).

`OperationSketch` fields: `name`, `dialect`, `dim_names`, `indexing_maps`, `iterator_types`, `compute_payload`, `num_inputs`, `num_outputs`, `description`. `Derived`: `num_loops`, `num_parallel`, `num_reduction`, `total_operands`, `parallel_dims()`, `reduction_dims()`.

Module-level collections: `SKETCH_LIBRARY` (ordered list of 35), `SKETCH_BY_NAME` (dict), `LINALG_SKETCHES` (22), `STABLEHLO_SKETCHES` (13). The full table is in Part 3.

Tracer output (`sympy_tracer.py`)

`TraceResult`: `output_expr` (SymPy expression for one output element), `index_syms`, `reduction_syms`, `tensor_syms` (IndexedBase per tensor), `matched_sketch`, `confidence`, `notes`.

Verifier output (`z3_checker.py`)

`CheckResult` (enum): `EQUIVALENT`, `NOT_EQUIVALENT`, `TIMEOUT`, `UNKNOWN`, `STRUCTURAL_MISMATCH`, `ENCODE_ERROR`.

`ReductionPatternKind` (enum, diagnostics only): `CONCRETE_UNROLL`, `SINGLE_SYMBOLIC_INFERRED`, `MULTI_SYMBOLIC_INFERRED`, `SINGLE_SYMBOLIC_UNINFERRED`, `MULTI_SYMBOLIC_UNINFERRED`.

`VerificationReport`:

Field	Meaning
<code>result</code> , <code>sketch_name</code> , <code>elapsed_ms</code>	Outcome and timing
<code>z3_model</code>	Full model text when a counterexample was found
<code>counterexample_bindings</code>	Dict of declaration name to value string
<code>failed_implication</code>	<code>source_to_sketch</code> or <code>sketch_to_source</code>
<code>mismatch_summary</code>	First six bindings as text
<code>sympy_confidence</code> , <code>sympy_z3_disagreement</code>	Set by the lifter; disagreement is <code>HIGH_CONFIDENCE_UNPROVED</code> or <code>HIGH_CONFIDENCE_REFUTED</code> when confidence is at least 0.8 but not proved
<code>notes</code>	Free text; carries <code>ENCODE_ERROR</code> details
<code>parametric_result</code> , <code>parametric_notes</code>	Filled only with <code>--parametric</code>
<code>proved</code> (property)	<code>result == EQUIVALENT</code>

Lifter output (`lifter.py`)

`PolicyProfile` (frozen): `name`, `strict_mode`, `z3_timeout_ms`.

`LiftResultState` (enum): `PROVED`, `UNPROVED_TIMEOUT`, `REFUTED`.

`LiftResult`:

Field or property	Meaning
<code>func_name</code> , <code>target_dialect</code> , <code>total_elapsed_ms</code>	Identification and timing
<code>matched_sketch</code> , <code>sketch_name</code>	Chosen sketch (may be set even when refuted)
<code>emitted_mlir</code>	Output text, or <code>None</code>
<code>verification</code>	The chosen candidate's report, or <code>None</code>
<code>sympy_confidence</code>	Confidence of the chosen candidate
<code>loop_info</code> , <code>candidates_tried</code> , <code>error</code> , <code>parser_diagnostics</code>	Context
<code>success</code>	sketch and emitted text and verification <code>EQUIVALENT</code>
<code>result_state</code>	<code>None</code> verification gives <code>REFUTED</code> ; <code>EQUIVALENT</code> gives <code>PROVED</code> ; <code>TIMEOUT/UNKNOWN</code> give <code>UNPROVED_TIMEOUT</code> ; anything else <code>REFUTED</code>
<code>partial_success</code>	sketch and emitted text and state <code>UNPROVED_TIMEOUT</code>
<code>is_accepted(strict)</code>	success, or <code>partial_success</code> when not strict
<code>summary()</code>	One-line text used by scripts and the demo

Frontend, validation, replay and corpus types

Class	Module	Fields
<code>CgeistInvocation</code>	<code>polygeist_frontend.py</code>	<code>source_files</code> , <code>language</code> (c/cpp), <code>function</code> , <code>include_dirs</code> , <code>defines</code> , <code>std</code> , <code>extra_clang_args</code> , <code>output_file</code> , <code>timeout_sec</code> , <code>cwd</code>
<code>CgeistResult</code>	<code>polygeist_frontend.py</code>	<code>command</code> , <code>returncode</code> , <code>elapsed_ms</code> , <code>stdout</code> , <code>stderr</code> , <code>mlir_text</code>
<code>PolygeistFrontendError</code>	<code>polygeist_frontend.py</code>	exception with <code>command</code> , <code>returncode</code> , <code>stderr</code> , <code>stdout</code>
<code>MlirValidationResult</code>	<code>mlir_validator.py</code>	<code>ok</code> , <code>command</code> , <code>returncode</code> , <code>stdout</code> , <code>stderr</code>
<code>CounterexampleReport</code>	<code>replay_checker.py</code>	<code>iv_values</code> , <code>source_access_pattern</code> , <code>sketch_access_pattern</code> , <code>parse_errors</code> , <code>diverging_cells</code> , <code>max_abs_diff</code> (always <code>None</code>), <code>verdict_summary</code>
<code>CorpusFileResult</code> , <code>CorpusCompatibilitySummary</code>	<code>parser_corpus_compat.py</code>	per-file compatibility classes and counts
<code>EmissionError</code>	<code>emitter.py</code>	<code>ValueError</code> subclass raised on missing or inconsistent metadata

Worked Example: Matrix Multiply End to End

This chapter traces the committed fixture `tests/fixtures/matmul.mlir` through every stage. All values below were captured from the real code in September 2026.

Input

```
func.func @matmul(%A: memref<4x4xf32>, %B: memref<4x4xf32>, %C: memref<4x4xf32>) {
  affine.for %i = 0 to 4 {
    affine.for %j = 0 to 4 {
      affine.for %k = 0 to 4 {
        %a = affine.load %A[%i, %k] : memref<4x4xf32>
        %b = affine.load %B[%k, %j] : memref<4x4xf32>
        %c = affine.load %C[%i, %j] : memref<4x4xf32>
        %mul = arith.mulf %a, %b : f32
        %add = arith.addf %c, %mul : f32
        affine.store %add, %C[%i, %j] : memref<4x4xf32>
      }
    }
  }
  return
}
```

Stage 1: extraction

The extractor scans the text line by line (after stripping `//` comments). It finds the function declaration, three `affine.for` lines, three loads, one store and two `arith` operations:

```
func_name      = matmul
induction_vars = ['i', 'j', 'k']
bounds         = {'i': (0, 4), 'j': (0, 4), 'k': (0, 4)}
tensor_shapes = {'A': [4, 4], 'B': [4, 4], 'C': [4, 4]}
element_type   = f32
reads          = %A[i, k] -> %a, %B[k, j] -> %b, %C[i, j] -> %c
writes         = %C[i, j] <- %add
compute_ops    = mulf(%a, %b) -> %mul, addf(%c, %mul) -> %add
parallel_vars  = ['i', 'j']          (they appear in the write index)
reduction_vars = ['k']              (it does not)
has_accumulation = True, accumulation_op = addf (C is read and written, and %add is stored)
```

Stage 2: candidate matching

For the `linalg` target the lifter iterates over the 22 Linalg sketches. `structural_match` keeps sketches with 3 loops, 1 reduction IV, and a payload compatible with "has multiply and add". Two survive:

`linalg.conv_1d_ncw_fcw` (3 loops `n`, `w`, `kw`, reduction `kw`) and `linalg.matmul`.

`SympyTracer.trace` builds a symbolic expression by substituting reads into the op chain, subtracting the old output value for accumulations and summing over `k = 0..3`:

```
A[i, 0]*B[0, j] + A[i, 1]*B[1, j] + A[i, 2]*B[2, j] + A[i, 3]*B[3, j]
```

Both candidates score confidence **1.00**. The scoring cannot tell a 1-D convolution from a matrix multiply because it looks at counts and payload categories, not at which indices appear where. The convolution recogniser returns nothing (no `+` in any index), so the tie is broken by library order and `linalg.conv_1d_ncw_fcw` is first.

Stage 3: verification

The lifter checks candidates in order. The convolution sketch's indexing maps do not fit (its input is indexed $(n, w + kw)$), so it is not proved, and the strided-convolution fallback does not apply because all coefficients are 1. Then `linalg.matmul` is checked.

For `matmul` the checker first aligns sketch dimensions to source IVs by role: parallel dims `m`, `n` map to `i`, `j` in order, reduction dim `k` maps to `k`. Every tensor becomes a Z3 uninterpreted function from integers to reals. Because all bounds are concrete and the reduction has 4 iterations (at most 32), the reduction is unrolled into an explicit sum. The two formulas built are:

```
source: ForAll([i, j], Implies(0 <= i < 4 and 0 <= j < 4,
    C(i, j) == 0 + A(i,0)*B(0,j) + A(i,1)*B(1,j) + A(i,2)*B(2,j) + A(i,3)*B(3,j)))
sketch: ForAll([i, j], Implies(0 <= i < 4 and 0 <= j < 4,
    C(i, j) == 0 + A(i,0)*B(0,j) + A(i,1)*B(1,j) + A(i,2)*B(2,j) + A(i,3)*B(3,j)))
```

Z3 is asked whether `not (source implies sketch)` is satisfiable, and then the reverse. Both are `unsat`, so the report is `EQUIVALENT` ("Both implications proved UNSAT - formally equivalent."), in about 15 ms.

Look at where the source formula came from

The source formula above was *not* built by following `%a`, `%b`, `%mul`, `%add`. It was built by inferring the payload category `MULTIPLY_ACCUMULATE` from the presence of `mulf` and `addf`, then multiplying "the first non-output read" by "the second". For this kernel that happens to coincide with the real computation. For `C += 2*A*B` or `C = B - A` it does not, and the proof still succeeds. See the verification architecture chapter and Part 5, F-01.

Stage 4: emission

`LinalgEmitter` looks up `linalg.matmul` in its handler table, checks that element type, input and output tensors and rank-2 shapes are known, and fills a template:

```
// LoopHole: Automatically lifted from scalar loop nest
// Source: matmul -> linalg.matmul
// Description: C[m,n] += A[m,k] * B[k,n] - dense matrix multiplication
// Generated by LoopHole POC v0.1.0

module {
  func.func @lifted_matmul(%A: memref<4x4xf32>, %B: memref<4x4xf32>, %C: memref<4x4xf32>) {
    linalg.matmul
      ins(%A, %B : memref<4x4xf32>, memref<4x4xf32>)
      outs(%C : memref<4x4xf32>)
    return
  }
}
```

`mlir-opt 17` accepts this. For the StableHLO target the same kernel becomes a function returning a fresh tensor, `%result = stablehlo.dot_general %A, %B, contracting_dims = [1] x [0]`, which drops the old contents of `c` (Part 5, F-07) and cannot be validated with the image's `mlir-opt`.

The same kernel from real C code

`loophole compile examples/smoke_matmul.c` produces the following (the long `dlti` module attribute is shortened):

```
module attributes {dlti.dl_spec = ...} {
  func.func @matmul_2x2(%arg0: memref<2x2xf32>, %arg1: memref<2x2xf32>, %arg2: memref<2x2xf32>) attribut
    es {llvm.linkage = ...} {
      %cst = arith.constant 0.000000e+00 : f32
      affine.for %arg3 = 0 to 2 {
        affine.for %arg4 = 0 to 2 {
          %0 = affine.for %arg5 = 0 to 2 iter_args(%arg6 = %cst) -> (f32) {
            %1 = affine.load %arg0[%arg3, %arg5] : memref<2x2xf32>
            %2 = affine.load %arg1[%arg5, %arg4] : memref<2x2xf32>
            %3 = arith.mulf %1, %2 : f32
            %4 = arith.addf %arg6, %3 : f32
            affine.yield %4 : f32
          }
          affine.store %0, %arg2[%arg3, %arg4] : memref<2x2xf32>
        }
      }
      return
    }
}
```

Differences from the fixture: numbered SSA names, the accumulator is a loop block argument (`iter_args`) rather than a load of `C`, and the result leaves the inner loop through `affine.yield`. The parser does not model `iter_args`; the kernel still lifts to `linalg.matmul` (PROVED, 8 ms) because the store index lacks `arg5` (so it is a reduction) and the op set contains a multiply and an add. That the right answer comes out is again a consequence of the category-based encoding, not of the parser understanding the accumulator.

Lifter Decision Flow and Result States

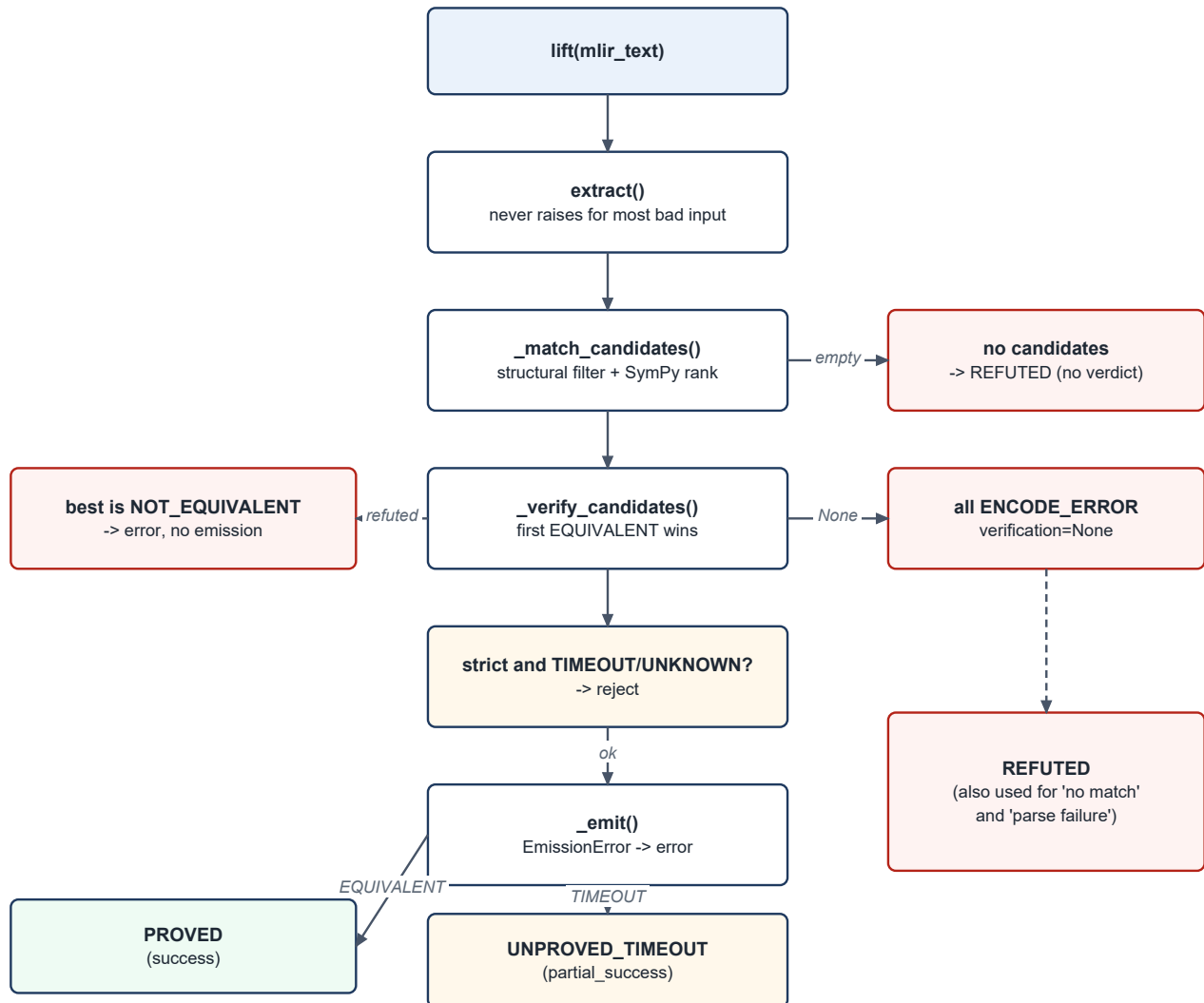


Figure: decision flow inside `Lifter.lift()` (`lifter.py:363-512`) and how each exit maps to a result state.

Candidate verification order

`_verify_candidates` walks the first `top_k` (default 3) candidates in ranking order:

1. Run `Z3EquivalenceChecker.check(loop, sketch)` and annotate SymPy/Z3 disagreement.
2. If `EQUIVALENT`, return immediately with this candidate.
3. If `TIMEOUT` or `UNKNOWN`, remember the first such candidate as the timeout fallback.
4. If `NOT_EQUIVALENT` and the sketch is a convolution, build a strided variant (coefficients from the input index expressions) and check again; an `EQUIVALENT` result returns the *original* sketch with the strided report. Remember the first refuted candidate.
5. `ENCODE_ERROR` and `STRUCTURAL_MISMATCH` are ignored.
6. After the loop return the timeout fallback if any, otherwise the refuted candidate, otherwise `None`.

Because the first `EQUIVALENT` wins, **a wrong sketch that proves beats a right sketch that is refuted**. Combined with an unsound encoding, this ordering is how the `elementwise_mul` fixture ends up as a scale operation (Part 5, F-02).

Acceptance and exit codes

Situation	result_state	success	partial_success	lift_exit	batch_counts	batch_exit (strict)	demo (strict)
EQUIVALENT and emitted	PROVED	True	False	0	proved	0	ok
TIMEOUT/UNKNOWN, not strict, emitted	UNPROVED_TIMEOUT	False	True	0	unproved_timeout	n/a	n/a
TIMEOUT/UNKNOWN, strict	UNPROVED_TIMEOUT	False	False	1 (error set)	unproved_timeout	1	exit 1
NOT_EQUIVALENT best candidate	REFUTED	False	False	1	refuted	1	exit 1
No candidates or all ENCODE_ERROR	REFUTED	False	False	1	refuted	1	exit 1
Emission error after a proof	PROVED	False	False	1	proved (no artifact)	0	ok if not refuted
Artifact validation failed	any	any	any	n/a	validation_fails	1 (always)	n/a

Two contract problems stand out. A lift that was proved but could not be emitted counts as PROVED in batch totals. The code path that makes `lift` exit 2 for a strict unproved result is effectively unreachable, because the lifter already rejected it and set an error (Part 5, F-13).

Configuration: Profiles, Flags and Environment

Policy profiles

Profile	Alias	<code>strict_mode</code>	<code>z3_timeout_ms</code>	Extra behaviour
<code>local-explore</code> (default)	<code>exploratory</code>	False	10,000	none
<code>ci-strict</code>	<code>trusted</code>	True	15,000	<code>batch</code> forces <code>--validate-emitted</code>

Resolution (`resolve_policy_profile`): explicit `--profile` unless it is `default`; else `LOOPHOLE_POLICY_PROFILE`; else `local-explore`. Aliases are applied after resolution. Explicit flags override profile values in `Lifter.from_policy_profile`, but the CLI's own defaults interfere: `lift` and `lift-c` always pass `--z3-timeout` (default 10,000), so the `ci-strict` 15,000 ms timeout never applies to them, while `batch` and `demo` pass `None` and do get it.

Environment variables

Variable	Read by	Effect
<code>LOOPHOLE_POLICY_PROFILE</code>	<code>lifter.resolve_policy_profile</code> , <code>conftest.py</code>	Default profile; <code>ci-strict/trusted</code> also makes artifact validation mandatory in tests
<code>LOOPHOLE_CGEIST_DOCKER_IMAGE</code>	<code>polygeist_frontend</code>	Image for <code>cgeist</code> and <code>lift-c</code>
<code>LOOPHOLE_MLIR_DOCKER_IMAGE</code>	<code>mlir_validator</code> , and <code>polygeist_frontend</code> as a fallback	Image for Docker <code>mlir-opt</code>
<code>LOOPHOLE_DISABLE_DOCKER_CGEIST</code>	<code>polygeist_frontend</code>	<code>1/true/yes</code> disables the Docker cgeist path
<code>LOOPHOLE_DISABLE_DOCKER_MLIR_VERIFY</code>	<code>mlir_validator</code>	Disables the Docker verifier; falls through to <code>PATH</code> , then to the internal fake verifier
<code>LOOPHOLE_MLIR_VERIFY_CMD</code>	<code>mlir_validator</code>	Explicit verifier command (first word must be on <code>PATH</code>)
<code>LOOPHOLE_REQUIRE_MLIR_VERIFY</code>	<code>conftest.py</code> only	Tests fail instead of skipping when no external verifier exists
<code>LOOPHOLE_DOCKER_IMAGE</code>	<code>cli.batch</code> report metadata only	Recorded in <code>run_metadata.docker_image</code> ; set by the PowerShell wrappers
<code>LOOPHOLE_DEMO_Z3_TIMEOUT_MS</code> , <code>LOOPHOLE_DEMO_FIXTURE_LIMIT</code>	<code>examples/run_demo.py</code>	Speed up the standalone demo
<code>PYTHONUTF8</code>	Python	Needed on Windows to avoid <code>charmap</code> decode errors on Docker output

There is no single configuration object. Profile, flags and variables are resolved separately in `cli.py` (per command), `lifter.py`, `mlir_validator.py`, `polygeist_frontend.py` and `conftest.py`, which is why defaults drift between commands.

Verification Architecture

Overall encoding

`Z3EquivalenceChecker.check(loop, sketch):`

1. Resets the per-check symbol cache and runs `structural_match` again (returns `STRUCTURAL_MISMATCH` if it fails).
2. Calls `_verify`, which picks `_verify_concrete` if every bound is an integer, else `_verify_symbolic`. `_verify_symbolic` **just calls** `_verify_concrete` inside exception handlers that convert unsupported reduction forms to `ENCODE_ERROR` and `ValueError` to `UNKNOWN`. There is one encoding path.
3. `_verify_concrete` creates one uninterpreted function `Intrank -> Real` per tensor (or a `Real` constant for rank 0), one `Int` per IV, then builds:
 - **source assertions** (`_build_source_assertions`): `ForAll(parallel IVs, bounds implies out(write index) == rhs)`, or a plain equality when there are no parallel IVs;
 - **sketch assertions** (`_build_sketch_assertions`): the same shape using the sketch's indexing maps after aligning sketch dimensions to source IVs;
 - **domain assumptions** for symbolic bounds (`_build_symbolic_shape_assumptions`): for each access of the form `iv + c` into a dimension of static extent `E`, assume `lo + c >= 0` and `hi + c <= E`.
4. Checks `not(assumptions and source implies assumptions and sketch)` and the reverse, each with the configured timeout. `sat` gives `NOT_EQUIVALENT` with a model; `unknown` gives `TIMEOUT`; anything else non-`unsat` gives `UNKNOWN`.

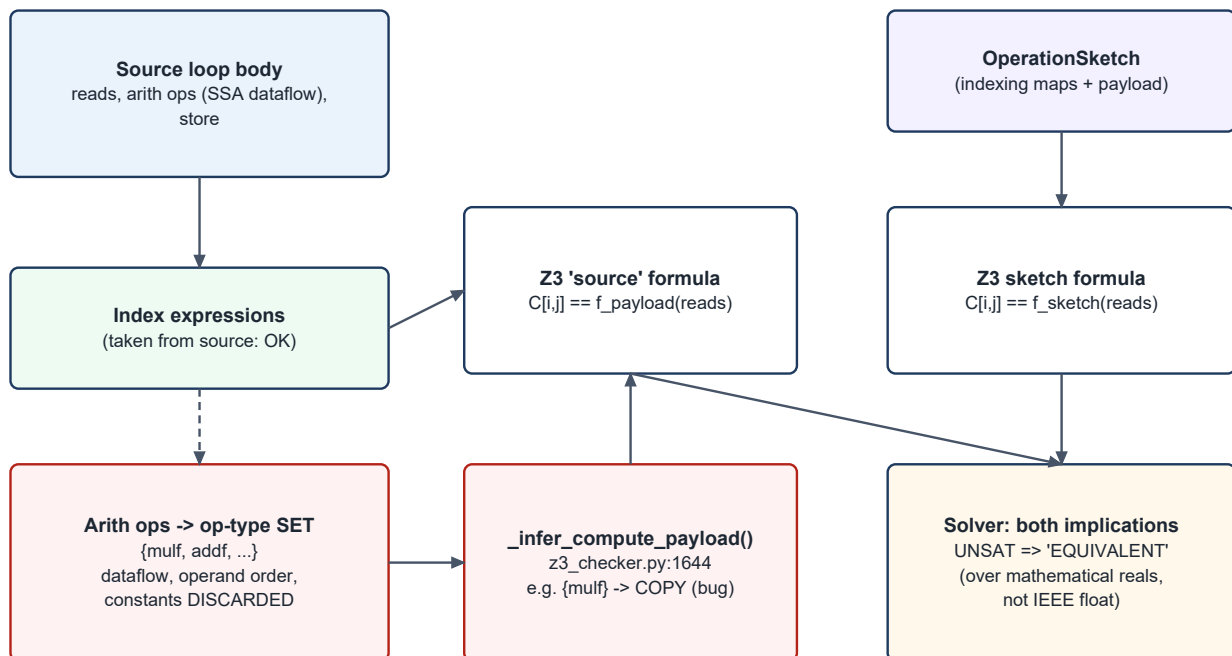


Figure: what the Z3 check really compares. Only the index expressions come from the source program; the arithmetic on the source side is re-synthesised from a coarse 'payload' guess, so arithmetic mistakes are invisible to the proof.

How the right-hand side is built

This is the heart of the soundness issue and worth understanding precisely.

- **Source side.** `_build_rhs_expr` calls `_infer_compute_payload(loop)`, which looks only at the set of `op_type` strings: multiply and add with reductions gives `MULTIPLY_ACCUMULATE`, multiply and add without gives `MULTIPLY`, add gives `ACCUMULATE_ADD/ADD`, subtract gives `SUBTRACT`, max gives `ACCUMULATE_MAX/MAX`, everything else `COPY`. The expression is then that payload applied to the first one or two reads that are not the output tensor, in textual order. Constants, the order of operands inside `subf`, additional reads, `divf`, `negf` and the actual SSA dataflow are never consulted.
- **Sketch side.** `_build_sketch_rhs` applies the sketch's declared payload to the operands selected by its indexing maps. Operand *i* of the sketch is bound to the *i*-th unique non-output read tensor of the source.

So both sides are "payload applied to reads at indices"; they differ only in which indices and which payload label. The proof really establishes "the index pattern of the source matches the sketch, assuming the source computes what its op-type set suggests".

Reduction strategies

Strategy	Used when	How	Limits
Concrete unroll (<code>_unrolled_reduction</code>)	All reduction bounds are integers and each range is at most <code>unroll_threshold</code> (32)	Python loop builds <code>term(k=lo) + ... + term(k=hi-1)</code>	Product of ranges grows fast (a 3x3x2 conv is 18 terms per output)
Guarded symbolic unroll (single IV)	One reduction IV, symbolic upper bound, maximum inferable from a static tensor extent	Sum up to the inferred maximum with <code>If(k < hi, total + term, total)</code>	Proves only for bounds up to the static extent
Guarded multi-reduction unroll (A-13)	Several reduction IVs, each upper bound inferable, total iterations at most 32 squared	Cartesian product with conjunctive guards	Same; cartesian blow-up guard
Recursive function (<code>_recfunc_reduction</code>)	Single reduction IV, not inferable or larger than 32	<code>RecFunction</code> with <code>acc(k) = If(k == lo, 0, acc(k-1) + body(k-1))</code>	Name collision across checks makes the second use fail (F-06); conv sketch arity bug; multi-reduction raises <code>_UnsupportedReductionForm</code>

Max reductions start from the constant `-1e30` instead of a true minus infinity (F-07).

Dimension alignment

`_align_dims` maps sketch parallel dimensions to source parallel IVs *in order*, and likewise for reduction dimensions. There is no search over permutations. If a source loop nest is written `j, i, k` (outer loop over columns), alignment maps `m` to `j` and the proof fails or, for symmetric shapes, may succeed with a transposed meaning. Loop order sensitivity is therefore part of the matching behaviour.

Parametric proof pilot (A-15)

With `--parametric`, after a concrete proof the lifter calls `_verify_parametric`: every distinct concrete upper bound value (at most 32) is replaced by a symbol (`M`, `N`, `K`, ...) and `_verify_concrete` runs again, which then takes the guarded-unroll paths. Equal bounds share a symbol and tensor shapes stay concrete, so the result covers "all square sizes up to the fixture size" rather than "all shapes" (F-14).

Numeric semantics

All values are reals. There is no modelling of IEEE-754 (rounding, non-associativity, NaN, signed zero), of integer width or overflow, or of out-of-bounds reads. Any use of the word "proved" externally must carry that qualification.

Emission Architecture

Handler tables

Each emitter has a `_NAMED_OP_HANDLERS` dict from sketch name to method name. `emit()` validates metadata (element type present, tensor shapes present, enough input tensors, input and output shapes non-empty), calls the handler to produce the `func.func` text, and wraps it with a comment header and `module { ... }`. An unmapped Linalg sketch raises `EmissionError`; an unmapped StableHLO sketch raises as well. The `linalg.generic` fallback (`_emit_generic`) is therefore unreachable.

Types and semantics

Aspect	Linalg emitter	StableHLO emitter
Argument types	<code>memref<...></code> (copied from the source)	<code>tensor<...></code>
Output	Written into an <code>outs(...)</code> argument, function returns nothing	Returned as a fresh value, output buffer not an argument
Accumulation into existing output	Named ops like <code>linalg.matmul</code> add to <code>outs</code> (matches <code>C +=</code>)	Dropped: result ignores old <code>C</code>
Dynamic dims	<code>?</code> in <code>memref</code>	<code>?</code> in <code>tensor</code>
Convolution shapes	Rank 2 (1-D NCW), 2 (simple 2-D), 4 (NHWC, NCHW)	Reshaped to rank-4 NHWC/HWIO from (2,1,2), (2,2,2) or (4,4,4)
Attribute inference	Strides and dilations from linear coefficients of input index expressions (<code>_infer_linear_coeff</code> , SymPy or regex fallback)	Same helpers
Transpose permutation	From positions of IVs in the write index; validated against shapes	Same logic, duplicated

Validity

Only some templates produce valid MLIR for `mlir-opt 17`: `matmul`, `matvec`, `transpose`, `conv_2d`, `conv_2d_nhwc_hwc` and `copy` were accepted; `map`, `reduce`, `dot` and `conv_1d_ncw_fcw` were rejected; StableHLO cannot be checked with that tool. Handlers that no fixture reaches (`vecmat`, `batch_matmul`, `conv_1d_nwc_wcf`, `conv_2d_nchw_fchw`, `pooling_nhwc_max`, `map` for `relu/scale/1-D add`) are untested against a real parser. See the per-handler table in Part 3.

The missing link between sketch and emitter

The sketch defines what an operation means for verification. The emitter independently decides what text to write for the same name. Nothing checks that the emitted operation actually has the semantics encoded in the sketch (for example, that `linalg.conv_1d_ncw_fcw` indexes its input as `(n, w + kw)` with rank 2, which it does not). A structural improvement would be to validate emitted artifacts with the real toolchain and a numerical oracle for every sketch, as proposed in Part 5.

Frontend and Validation Architecture

Two separate paths into Polygeist

Aspect	<code>loophole_compile(tli.compile_cmd)</code>	<code>loophole_cgeist / lift-c (PolygeistFrontend)</code>
Invocation	<code>`docker run ... bash -lc "cgeist ...</code>	<code>sed ...</code>
Default image	<code>ghcr.io/schizoid-man/loophole-polygeist:11vm17</code> (hard-coded default)	explicit flag, then <code>LOOPHOLE_CGEIST_DOCKER_IMAGE</code> , then <code>LOOPHOLE_MLIR_DOCKER_IMAGE</code> , then first local image of <code>loophole-polygeist:11vm17</code> or the GHCR name
Function selection	Always <code>--function=*</code>	<code>--function</code> option
Post-processing	<code>sed</code> rewrites legacy <code>-1 dims</code> to <code>?</code> , then <code>mlir-opt --canonicalize</code>	None
Timeout	None	<code>--timeout-sec</code> (default 60)
Include dirs, defines, clang args	Not supported	Supported and mapped into the container
Mount	Common parent of source and output	Common parent of sources, includes and output

Both require all paths on one drive (single volume mount on Windows).

Verifier discovery

`find_mlir_verifier(preferred)`:

1. An explicit `--mlir-verifier` command if its executable is on PATH, else `None`.
2. `LOOPHOLE_MLIR_VERIFY_CMD` if its executable is on PATH.
3. The Docker verifier sentinel `loophole-docker-mlir-verify`, if Docker is running and the image exists (unless disabled by environment variable).
4. `mlir-opt`, `mlir-opt-18`, `mlir-opt-17` on PATH.
5. The internal sentinel `loophole-internal-mlir-verify`, a brace-and-keyword check.

`validate_mlir_artifact` dispatches on the sentinel: Docker runs `mlir-opt /work/artifact.mlir` in a container (timeout at least 30 s, errors captured); internal runs the text checks; anything else runs the command with the artifact path appended (a timeout here raises out of `batch`). There is no per-dialect selection: StableHLO goes to `mlir-opt`, which does not know it.

Module Dependency Graph

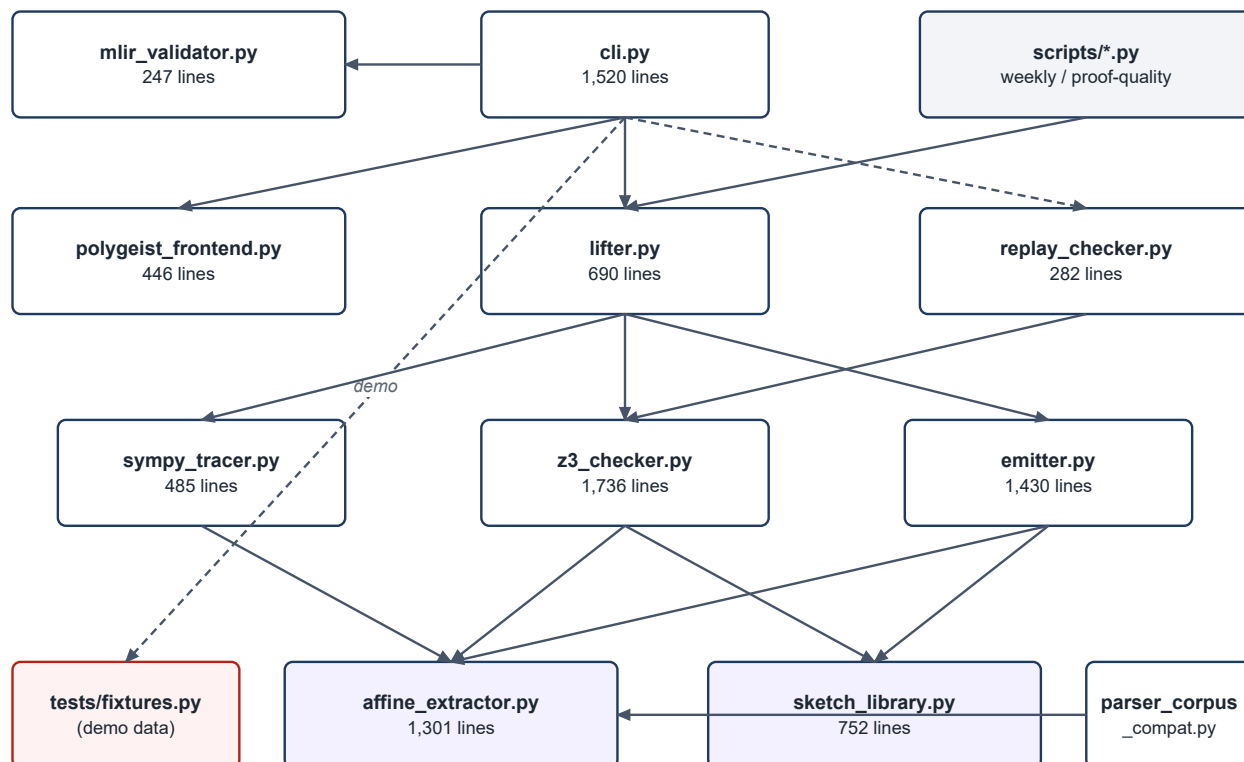


Figure: import graph between modules (arrow = 'imports'). `affine_extractor` and `sketch_library` are the leaves every other layer depends on.

Layering, from bottom to top:

- Leaves:** `affine_extractor.py` (data model and parser) and `sketch_library.py` (sketches). Neither imports other project modules.
- Engines:** `sympy_tracer.py`, `z3_checker.py` and `emitter.py`, each importing both leaves. `lifter.py` additionally imports a private helper `_infer_linear_coeff` from `emitter.py` for strided convolution sketches, which couples the orchestrator to emitter internals.
- Orchestration:** `lifter.py`; `replay_checker.py` (imports `z3_checker` types); `parser_corpus_compat.py` (imports the extractor).
- Surfaces:** `cli.py` (imports lifter, frontend, validator, sketch library, and lazily replay checker, extractor and test fixtures); `scripts` (import lifter and each other).
- Tooling adapters:** `polygeist_frontend.py` and `mlir_validator.py`, which only use the standard library.

There are no import cycles. The main coupling problems are the CLI's dependency on `loophole.tests.fixtures`, the lifter's import of an emitter private function, the script importing another script's private helpers (`generate_proof_quality_summary.py` imports underscore functions from `generate_weekly_benchmark_report.py`), and `conftest.py` sitting at package root outside `tests/`.

Architectural Drawbacks Summary

Drawback	Consequence	Part 5 reference
Parser output lacks steps, dataflow, loop nesting and function boundaries	Later stages cannot be correct even if they try	F-03, F-04, F-05, F-20, F-21
Verifier re-derives source semantics from op-type sets	False proofs	F-01, F-02, F-18
First-EQUIVALENT-wins ordering with heuristic ranking	Wrong sketch can beat right one	F-02, F-19
Sketch semantics and emitter templates not cross-checked	Invalid or mismatched artifacts	F-08
Text-based parsing and emission instead of MLIR APIs	Fragile to formatting and real frontend output	F-16, F-34
One verifier for all dialects, with a fake fallback	Gates that cannot fail	F-09, F-10
Result states conflate errors with refutations	Misleading metrics	F-12, F-28
Configuration resolved in many places	Inconsistent defaults	F-24
Two Polygeist invocation paths	Different outputs for the same source	F-23
Test data and demo data in the runtime package	Packaging and layering confusion	F-26